Fangorn I — The Mathematical Core (study sheet)

Condensed from WHITEPAPER_1.md. Definitions, algorithms, results; proofs as one-line ideas.

Flow: names → Merkle DAGs → map/log commitments → ordering → repos, rules, apps → registry → global state per context → directories & logs → reading slices & history → lenses → fraud proofs → search (bundles, bounded trees) → instantiation (MST, MMR) → agents.

1. Primitives

Notation. λ security parameter; enc canonical injective encoding; $F(x_1,\dots,x_n) := F(\mathsf{enc}(x_1,\dots,x_n))$. $K \rightharpoonup V$ finite partial map. fail propagates to callers. $\mathsf{Get}(\alpha)$: fetch from any source, accept iff its name is α.

Def 1.1 (Binding name). $N : \{0,1\}^* \to \{0,1\}^{2\lambda}$. $\mathbf{Adv}^{\mathsf{bind}}_N(\mathcal{A}) = \Pr[\mathcal{A} \to x \neq x',\ N(x) = N(x')]$ is negligible. (= collision resistance.)

Def 1.2 (Node). $n = (v, C)$, payload $v \in \{0,1\}^*$, links $C = (c_1,\dots,c_m)$ (ordered). Name $N(n) = N(v, C)$.

Def 1.3 (Merkle DAG). Finite node set $\mathcal{N}$ with (1) *closure*: every link names a node in $\mathcal{N}$; (2) *distinct names*.

- $n_\alpha < n_\beta$ (descends) if a path $n_\beta \to \cdots \to n_\alpha$ of ≥ 1 links exists.
- $\mathcal{N}{\downarrow}\alpha$: nodes reachable from n_α (itself a Merkle DAG, root α).

Lemma 1.4 (Acyclic; ROM). With q oracle queries, $\Pr[\text{cycle}] \leq q^2/2^{2\lambda}$. *Idea:* the first-queried node on a cycle contains the output of a later query. Needs unpredictability, not just binding.

Lemma 1.5 (A name identifies its sub-DAG). $\mathcal{N}{\downarrow}\alpha \neq \mathcal{N}'{\downarrow}\alpha \Rightarrow$ collision in $\mathcal{N} \cup \mathcal{N}'$. *Idea:* walk a path from α in both; the first point of divergence is two nodes with one name. ⇒ **content addressing**: fetching by name from untrusted hosts yields exactly $\mathcal{N}{\downarrow}\alpha$.

Def 1.6 (Map commitment Com). Finite map $\mapsto$ root of a deterministic Merkle DAG over its entries, with:

1. **binding:** $D \neq D' \Rightarrow \mathsf{Com}(D) \neq \mathsf{Com}(D')$ (w.h.p.);
2. **unique representation:** $\mathsf{Com}(D)$ depends only on D, not on edit history;
3. **interval openings:** keys totally ordered; $\mathsf{Open}(D, I) \to \pi$, $\mathsf{Check}(root, I, D', \pi) \in \{0,1\}$
   - correctness: $\mathsf{Check}(\mathsf{Com}(D), I, D|_I, \mathsf{Open}(D, I)) = 1$;
   - opening binding: no two accepted $D' \neq D''$ for one $(root, I)$, except $\mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}$.
   - $[k, k]$ also proves absence.

Def 1.7 (Log commitment LCom). Sequence $L = (l_1,\dots,l_M) \mapsto$ root, with:

1. **binding** (root fixes M);
2. **positional openings** $\mathsf{Prove}(L, j)$, $\mathsf{Verify}(root, j, l, \pi)$; position binding except $\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}}$;
3. **incremental appends:** small state $\mathsf{st}(L)$ fixing M; $\mathsf{Root}(\mathsf{st}(L)) = \mathsf{LCom}(L)$, $\mathsf{Append}(\mathsf{st}(L), l) = \mathsf{st}(L \| l)$.

Follow an untrusted log: hold st, append received entries, compare Root to an authenticated root ⇒ all entries, in order.

Def 1.8 (NP relation). $R \subseteq \{0,1\}^* \times \{0,1\}^*$ with $|w| \leq p(|x|)$ and $(x, w) \in R$ decidable in $\mathrm{poly}(|x|)$. $\mathcal{L}_R = \{x : \exists w, (x, w) \in R\}$. (Used for validation rules ν_V, acceptance rules α_A and exclusion rules χ_A: statement = one claim or change; witness = evidence or authorization.)

Def 1.9 (Merkle clock). Merkle DAG of events; each new node links to all current roots. $a \prec b$ iff $n_a < n_b$. Gives Lamport happened-before; one writer ⇒ a chain.

Def 1.10 (Ordering service). Puts everyone's updates into one total order, with a logical clock. It keeps a sequence $\mathcal{O} = (o_1, o_2, \dots)$ of entries $o_i = (t_i, a_i, d_i, op_i)$:

- t_i clock value (assigned by the service), a_i author, d_i **destination** (the registry it is addressed to, §3), op_i operation (both chosen by the author).
- Interface: $\mathsf{Submit}(o)$ proposes an entry; $\mathsf{Read}(t)$ returns the prefix up to clock t; $\mathsf{Verify}(d, \mathcal{O}')$ checks that $\mathcal{O}'$ validly extends checkpoint d.
- Epoch $e(t) = \lfloor t/\eta \rfloor$ for a fixed length η.

Assumption O (Ordering).

1. **Agreement:** all honest parties see the same sequence, except for entries less than δ clock steps old (the finality delay).
2. **Authorship:** an efficient adversary outputs an accepted entry (t, a, d, op) where a never signed (d, op) only with negligible probability. *(The signature covers the destination, so an entry can't be redirected to another registry.)*
3. **Clock:** clock values never decrease along the sequence.

2. Data, rules, apps

- **Record** $r = (\theta, v)$: type, field map. **Version** $= N(\texttt{rec}, r)$.
- **State** $M : K \rightharpoonup Rec$, $\quad \mathsf{root}(M) = \mathsf{Com}\big(\{k \mapsto N(\texttt{rec}, M(k))\}\big)$.
- **Repo** $\rho = (P, s)$: one publisher's data. Only P writes it. It **belongs to no app and names no rule**.
- **Commit** $\gamma = (e, parent, root)$; **head** $= N(\gamma)$; M_h = state of head h, $M_\perp = \emptyset$. A repo's commits are a single-writer Merkle clock (a chain).
- **Validation rule** $V = (n_V, \nu_V)$, with $n_V = N(\texttt{rule}, \nu_V)$ (content-addressed: same rule ⇒ same name; nobody owns it). ν_V is an NP relation over "head h at position i of ρ satisfies V", statement $(\iota, V, \rho, i_0, i, h)$, with i_0 the position last validated (⇒ **incremental** evidence: "h passes given h_{i_0} passed").
  - What it checks is its own: schema, relations between records, format, or terms needing judgment.
  - Kinds of witness, each exactly as strong as its evidence: **the data itself** + openings, checked by the registry (as strong as the ordering); **a succinct proof** (as its proof system); **an attestation**: a signature from an attested-hardware or named validator key (as the hardware and signer).
- **App** $A = (n_A, o_A, \alpha_A, \chi_A)$: name, owner, **acceptance rule** α_A, **exclusion rule** χ_A. Both rules are NP relations that may read the ordering at apply time (⇒ revocable key lists).
  - An app **composes validated data**: it holds no data and dictates no schema. Typical α_A: "h is validated under the rules I name" + a selection (any such repo / publishers o_A lists / a curator's approval).
  - **Admission** is the special case α_A = "signed by a publisher o_A lists", with acceptance bundled in each update.
  - Apps requiring the same rule share its validations: **validate once, present in many apps**.

3. The registry and the global state

What it is. The ordering stores entries but doesn't interpret them. The **registry** is a deterministic state machine run over the ordering that gives them meaning: which rules and apps exist, every repo's head, what passed each rule, what each app presents, and what it has excluded. Anyone who holds the ordering can run it and gets the same state, so nobody has to be trusted to enforce the rules.

Parameters, fixed at deployment:

- an **identifier** ι, unique to this registry (on a chain: chain id + contract address);
- a **window** $W \geq 1$: how many epochs old a commit may be when it lands.

Which entries it reads. One ordering may serve many registries. Registry ι runs over $\mathcal{O}_\iota$, the entries with destination $d = \iota$, and ignores the rest. Inside one registry d is fixed, so we write entries as $o = (t, a, op)$. By Authorship, nobody can move an entry from one registry to another. Witnesses are signed outside any entry, so they carry ι inside their statement instead.

Def 3.1 (Registry state). $R = (\mathcal{R}, \mathcal{A}, \mathcal{H}, \mathcal{V}, \mathcal{D}, \mathcal{X}, \mathcal{C}, \mathcal{U}, \mathcal{L})$:

map	sends	to
$\mathcal{R}$	a rule's name n_V	the rule V
$\mathcal{A}$	an app's name n_A	the app tuple A
$\mathcal{H}$	a repo $\rho = (P, s)$	(pos, h): committed position and head. $(0, \perp)$ if never written
$\mathcal{V}$	(rule V, repo ρ)	(n, i, h): # validations, last **validated** position and head. $(0, 0, \perp)$ if none
$\mathcal{D}$	(app A, repo ρ)	(n, i, h): # acceptances, last **accepted** position and head. $(0, 0, \perp)$ if none
$\mathcal{X}$	an app	its current **exclusions**: a publisher P, a repo (P, s), or a key (P, s, k) — any, accepted or not. Covers every key under what it names
$\mathcal{C}$	(app, exclusion X)	# valid excludes + restores of X so far (0 if never)
$\mathcal{U}$	—	the global **update log**: every valid update, in order
$\mathcal{L}$	an app / a rule	its **app log** (acceptances, excludes, restores) / **rule log** (validations)

R_0 is all maps and logs empty.

The seven operations. $\mathsf{Apply}(R, o)$ runs one entry. Each operation has a **guard**. If it fails, the entry is **invalid** and R is unchanged.

- $\mathsf{rule}(V)$ — *by* anyone. *Requires* $n_V = N(\texttt{rule}, \nu_V)$, unused. *Effect:* registers V, empty rule log.
- $\mathsf{app}(A)$ — *by* the owner o_A. *Requires* n_A unused. *Effect:* creates A, no exclusions, empty app log.
- $\mathsf{update}(s, e, parent, root, Val, Acc)$ — *by* $P = a$, on its own repo (P, s) with $(i, h) = \mathcal{H}(P, s)$. *Requires* $parent = h$ and $e(t) - W \le e \le e(t)$. *Effect:* head $h' = N(e, parent, root)$, position $i{+}1$. *Logs* $(P, s, i{+}1, h', t)$ to $\mathcal{U}$. Then applies each validation in Val, then each acceptance in Acc, of h', each valid or not on its own. **Names no rule and no app**; the bundles are transport only.
- $\mathsf{validate}(V, P, s, i, h, w)$ — *by* anyone holding w. With $(n, i_0, \cdot) = \mathcal{V}(V, P, s)$: *requires* V exists, $i_0 < i$, $(P, s, i, h, \cdot) \in \mathcal{U}$ (so h is the head at i), and $((\iota, V, (P, s), i_0, i, h), w) \in \nu_V$. *Effect:* $\mathcal{V} \leftarrow (n{+}1, i, h)$. *Logs* $(\mathsf{validate}, P, s, n{+}1, i, h, t)$ to $\mathcal{L}(V)$. No deadline: a fact about fixed content.
- $\mathsf{accept}(A, P, s, i, h, t_x, w)$ — *by* anyone holding w. With $(n, i_0, \cdot) = \mathcal{D}(A, P, s)$: *requires* A exists, $i_0 < i$, $(P, s, i, h, \cdot) \in \mathcal{U}$, $t \le t_x$, and $((\iota, A, \mathsf{accept}, (P, s), i_0, i, h, t_x), w) \in \alpha_A$. *Effect:* $\mathcal{D} \leftarrow (n{+}1, i, h)$. *Logs* $(\mathsf{accept}, P, s, n{+}1, i, h, t)$ to $\mathcal{L}(A)$.
- $\mathsf{exclude}(A, X, t_x, w)$ — *by* anyone holding w. *Requires* A exists, $X \notin \mathcal{X}(A)$, $t \le t_x$, and w authorizes σ (below) under χ_A. *Effect:* adds X; $\mathcal{C}(A, X)$ += 1. *Logs* $(\mathsf{exclude}, X, \mathcal{C}(A, X), t)$.
- $\mathsf{restore}(A, X, t_x, w)$ — mirror image: *requires* $X \in \mathcal{X}(A)$; removes it; $\mathcal{C}(A, X)$ += 1. *Logs* $(\mathsf{restore}, X, \mathcal{C}(A, X), t)$.

Reading the guards.

- *Update — compare-and-swap:* applies only if its parent is the head. Heads never repeat (Thm 3), so a signed update applies **at most once**, even if the repo returns to an old state.
- *Update — epoch window:* up to W epochs of inclusion lag; a late update is invalid and the publisher re-commits.
- *Update — no gate:* any author commits to its own repos at its own cost; an update alone changes what no app presents.
- *Validate / accept — forward only:* each moves from i_0 to a later i, may **skip** positions (a failing head is never validated; an app can lag a fast repo), never moves back. h is pinned by $\mathcal{U}$, so the witness binds the exact content approved. Naming $i_0 \Rightarrow$ one witness, one step, once.
- *Who pays:* whoever submits. An app can **pull** (pay to accept data published for free); a publisher can **push** (bundle into its update). A validation is paid once and serves every app requiring its rule.
- *No fetching:* Apply computes heads from commit fields; witnesses carry whatever their checks need.

Witnesses for exclusions. Statement

$$\sigma = (\iota,\ A,\ op,\ X,\ k,\ t_x), \qquad k = \mathcal{C}(A, X)$$

— registry, app, direction, target, prior change count, deadline. One witness authorizes **one change, in one direction, once, in one registry**; it goes **stale** if X changes first, the deadline passes, or χ_A stops accepting it; binding to the per-target count means unrelated changes don't stale it. An exclusion may name data the app never accepted (refuse before offered).

Cost of Apply. Constant map changes + ≤ 1 log append (an update also applies its bundles), plus checking w under ν_V, α_A or χ_A.

The registry at t. R_t = Apply folded over $\mathsf{Read}_\iota(t)$ from R_0. Read off it: $H_t(\rho), pos_t(\rho)$ (committed); $H_t(V, \rho), pos_t(V, \rho), n_t(V, \rho)$ (validated); $H_t(A, \rho), pos_t(A, \rho), n_t(A, \rho)$ (accepted); $\mathcal{X}_t(A)$, $\mathcal{C}_t(A, X)$; logs $\mathcal{U}_t, \mathcal{L}_{A,t}, \mathcal{L}_{V,t}$. **Validity is a function of the ordering alone.** A repo's **history** is the heads its updates set; its **validated history** under V and **accepted history** in A are subsequences of it.

Global states, per context. A **context** c is an app A or $\perp$ (the publisher's own word).

- **Committed:** $\hat{\mathcal{G}}_t(\rho, k) = M_{H_t(\rho)}(k)$. **Accepted by A:** $\hat{\mathcal{G}}^A_t(\rho, k) = M_{H_t(A,\rho)}(k)$.
- **Presented**, reading at t the state as of $t' \le t$, for $\rho = (P, s)$:

$$\mathcal{G}^t_{t'}[A](\rho, k) = \begin{cases} \hat{\mathcal{G}}^A_{t'}(\rho, k) & \text{if no } X \in \mathcal{X}_t(A) \text{ covers } (P, s, k) \\ \text{undefined} & \text{otherwise} \end{cases}$$

and $\mathcal{G}_t[c] := \mathcal{G}^t_t[c]$.

Data from t', exclusions from $t \Rightarrow$ exclusions are **retroactive**. An exclusion changes only what **one app** presents: not the repo, not its validations, not other apps. $\mathcal{G}_t[c]$ needs no root: a checkpoint at t names $\mathsf{Read}(t)$.

Prop 3.2 (Exclusions are explicit). Every change to $\mathcal{X}(A)$ is a valid entry with a witness for exactly that change. For the valid exclude/restore entries $o_1, o_2, \ldots$ of one X: o_j is an exclude iff j odd; its witness is for count $j - 1$, its log entry carries j; no statement is accepted twice. So $X \in \mathcal{X}(A) \iff \mathcal{C}(A, X)$ odd: **the counter is also the flag.** *Idea:* only exclude/restore touch $\mathcal{X}, \mathcal{C}$; each adds 1 and flips membership; induct. Caveat: binding to one change needs the rule to read the whole statement (same for α_A, ν_V and i_0, h).

4. Directories and logs

Why. A reader shouldn't replay the ordering to find heads. **Directories** map names to heads, committed as trees, so a reader opens only what it needs.

Def 3.3 (Directory). A state whose records are (e, c), not both $\perp$: e a repo entry (a head plus the counts locating it in a log), $c = \mathsf{Com}(D')$ a subdirectory root. Directories nest; a path is a sequence of keys.

Each tree's keys are chosen by exactly one party (§10, shape):

- **publisher directory** (keys chosen by P): $D_{P,t}(s) = \big(pos_t(\rho), H_t(\rho)\big)$, root $\mathsf{dir}_{P,t}$;
- **app directory**, two levels: $D_{A,t}(P) = \big(\bot, \mathsf{Com}(D_{A,P,t})\big)$ (keys chosen by whoever α_A lets accept), $D_{A,P,t}(s) = \big(n_t(A,\rho), pos_t(A,\rho), H_t(A,\rho)\big)$ (keys chosen by P), root $\mathsf{dir}_{A,t}$;
- **no directory of every publisher** (everyone would choose its keys): reads in context $\bot$ name one publisher; many publishers ⇒ via an app or a lens over $\mathcal{U}$. Repo names may be paths $s = (s_1, \ldots, s_d)$, compared lexicographically ⇒ prefixes are intervals; lookup ≤ $(d+1)(h+1)$ nodes.

Lemma 3.4 (Directories only grow). $t \le t' \Rightarrow pos_t(\rho) \le pos_{t'}(\rho)$ and $n_t(A,\rho) \le n_{t'}(A,\rho)$; equal position (resp. count) ⇒ equal entries. ⇒ merge by max.

Logs. All committed as LCom; directories are folds of them.

- **Update log** $\mathcal{U}_t$: (P, s, i, h, t_u) per valid update. Root log_t. Folds to publisher directories.
- **App log** $\mathcal{L}_{A,t}$: $(\mathsf{accept}, P, s, n, i, h, t_u)$ and (op, X, j, t_u). Root $\mathsf{log}_{A,t}$. Folds to app directories. Records what the **app** did, not every update of its publishers.
- **Rule log** $\mathcal{L}_{V,t}$: $(\mathsf{validate}, P, s, n, i, h, t_u)$. Root $\mathsf{log}_{V,t}$. Lists every repo that passed V ⇒ discovery by rule.

Lemma 3.5 (Logs locate every head). For each ρ: exactly one $\mathcal{U}$ entry per position $1..pos_t(\rho)$, increasing. For each (A,ρ): one acceptance entry per count $1..n_t(A,\rho)$, increasing, with increasing positions; same for (V,ρ) in $\mathcal{L}_{V,t}$. For each X: counts $1..\mathcal{C}_t(A,X)$. Clocks non-decreasing.

Cost per valid update / validation / acceptance: ≤ $2(h+1)$ node rewrites per directory tree on its path (L6.1); log append < 2 hashes avg, ≤ $\lfloor \log_2 M \rfloor$ (L6.3).

5. Reading

Def 3.6 (Selector, slice). S = finite union of boxes $\{c\} \times I_P \times I_s \times I_k$; in context $\bot$, $I_P = [P,P]$. Slice $\mathcal{G}_t|_S$ = union over boxes of $\mathcal{G}_t[c]$ restricted to the box. Boxes may mix contexts (one repo as two apps present it, or as committed). (Intervals because openings are on intervals; slicing by field = slice of an index view.)

Def 3.7 (Authenticated directories & logs). Light reader holds checkpoint d at final t. For context c and box $B = I_P \times I_s$:

1. $\mathsf{Dir}(d,c,B)$: $\{(P,s) \mapsto D_{A,P,t}(s)\}$ for $c = A$; $\{(P,s) \mapsto D_{P,t}(s)\}$ for $c = \bot$ — every repo in B with a head in c, nothing else;
2. $\mathsf{Log}(d,c) = \mathsf{st}(\mathcal{L}_{A,t})$ for $c = A$, $\mathsf{st}(\mathcal{U}_t)$ for $c = \bot$;
3. $\mathsf{Excl}(d,A,Y) = \{X \mapsto \mathcal{C}_t(A,X)\}_{X \in Y}$ (odd = excluded). Key (P,s,k) is covered only by P, (P,s), (P,s,k) — no global list needed.

Wrong non-fail answer w.p. ≤ $\mathbf{Adv}^{\mathsf{dir}}$ (= 0 for a full reader).

$\mathsf{ReadSlice}(d, S)$

```
1  Z ← ∅
2  for each box {c}×I_P×I_s×I_k of S:
3     E ← Dir(d,c,I_P×I_s)
      if c = ⊥: X ← ∅  else: Y ← {P,(P,s) : (P,s)∈dom E};  X ←
{y∈Y : Excl(d,c,Y)(y) odd}
4     drop (P,s) from E if P∈X or (P,s)∈X
5     for each (P,s) ∈ dom E  (parallel):
6       h ← head of E(P,s);  (e,parent,root) ← Get(h)
7       (D',π) ← opening of root on I_k;  require
Check(root,I_k,D',π)
8       for k↦n ∈ D' with c = ⊥ or Excl(d,c,{(P,s,k)}) even:
Z((c,P,s),k) ← Get(n)
9  return Z
```

Cost (r repos, m records): r commits; $r(h+1) + m/(2^b - 1)$ nodes expected ($r(h+1) + mh$ worst); m records; **4 round trips** regardless of size.

History. A *track* = log entries for one subject, indices $1..n$ in order, clocks non-decreasing:

- repo ρ in $\mathcal{U}$: index i, value h;
- repo ρ in $\mathcal{L}(A)$ (or $\mathcal{L}(V)$): index n, value h;
- exclusion X in $\mathcal{L}(A)$: index = value j.

$\mathsf{LastAt}(\ell, n, \sigma, t')$ — last entry of track σ with clock ≤ t'

```
1  if n = 0: return ⊥
2  (j,l,π) from any source; require Verify(ℓ,j,l,π) ∧ l ∈ σ with
index i, value v, clock t_u
3  if t_u > t': require i = 1; return ⊥
4  if i < n: (j',l',π') ; require Verify(ℓ,j',l',π') ∧ l' ∈ σ,
index i+1, clock > t'
5  return v
```

- $\mathsf{ReadAt}(d,c,\rho,t')$: ℓ from $\mathsf{Log}(d,c)$, n from ρ's entry in $\mathsf{Dir}(d,c,\cdot)$ (position for $\bot$, count for A) ⇒ $H_{t'}(\rho)$ or $H_{t'}(A,\rho)$.
- $\mathsf{ExclAt}(d,A,X,t')$: $n = \mathcal{C}_t(A,X)$ from Excl; excluded iff result odd. A key's status at t': ≤ 6 openings.

Prop 3.8 (History reads sound & complete). LastAt errs w.p. ≤ $2\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}}$; $\mathsf{ReadAt}/\mathsf{ExclAt}$ ≤ $2\mathbf{Adv}^{\mathsf{dir}} + 2\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}}$; honest source never fails. *Idea:* position binding + L3.5 ⇒ entry i at/before t' and entry $i+1$ after ⇒ i is last.

Follow (keeping up with changes). Cursor (c, B, I_k, st, E, X): context, box, key range, log state, repo entries, exclusions.

1. Fetch the context's new log entries $M{+}1..M'$, append to st, require $\mathsf{Root}(st) = \ell'$ ⇒ every new entry, in order.
2. Replay entries touching B: new heads (updates in $\bot$, acceptances in A), exclusions/restores.
3. For each changed repo, Diff old and new state trees: skip equal-named subtrees and subtrees outside I_k. Cost: Δ_c entries, ≤ $2\Delta_c + \lfloor \log_2 M \rfloor + 1$ hashes; $2c'$ commits (c' changed repos); $O((\Delta + c' + x)(h+1))$ nodes; Δ records. **Independent of slice/app size. Ceiling:** following an app reads only its own log (accepts + exclusions); following one publisher in $\bot$ reads all of $\mathcal{U}$ — or an indexer's sub-log.

6. Security of data and history

Thm 1 (Binding). Reader accepts only M_h for head h, except $\mathbf{Adv}^{\mathsf{bind}}_N + \mathbf{Adv}^{\mathsf{bind}}_{\mathsf{Com}}$.

Thm 2 (Authenticity, validation, acceptance).

1. Every head of (P,s) was set by an entry authored by P.
2. Every head in its accepted history in A is the head at its position, accepted with a witness under α_A (as evaluated then) for that step.
3. Every head in its validated history under V is the head at its position, with a witness under ν_V for that step. *(guards + Authorship + L3.5.)* A validation shows exactly what its evidence shows; an acceptance whatever α_A demands — typically validations.

Thm 3 (Linear history). Valid updates of a repo form one sequence; the i-th has parent h_{i-1}; no head repeats except $\mathbf{Adv}^{\mathsf{bind}}_N$. Acceptances (per app) and validations (per rule) each form one sequence with increasing positions ⇒ subsequences of the history. *Idea:* least j with $h_i = h_j$: equal commits ⇒ $h_{i-1} = h_{j-1}$, contradiction; unequal ⇒ collision. ⇒ a signed update applies at most once.

Thm 4 (Slice soundness). t final, q boxes, r repos:

$$\Pr[\mathsf{ReadSlice}(d,S) \notin \{\mathcal{G}_t|_S, \mathsf{fail}\}] \le (2q+r)\,\mathbf{Adv}^{\mathsf{dir}} + \mathbf{Adv}^{\mathsf{bind}}_N + \mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}.$$

Idea: lines 3–4 true by $\mathsf{Dir}/\mathsf{Excl}$ (none in $\bot$); line 6 by name; line 7 by opening binding; line 8 by Excl + name. **Completeness, not just inclusion:**

hosts can withhold (⇒ fail), not truncate.

Cor 4.1 (Historic slices). Lines 3–4 at t + LastAt per repo (its track in the context's log) + lines 6–8 ⇒ $\mathcal{G}^t_{t'}|_S$, except $(3q+r)\mathbf{Adv}^{\mathsf{dir}} + 2r\,\mathbf{Adv}^{\mathsf{pos}}_{\mathsf{LCom}} + \mathbf{Adv}^{\mathsf{bind}}_{N} + \mathbf{Adv}^{\mathsf{open}}_{\mathsf{Com}}$.

7. Lenses (views)

Def 3.9 (Lens). $L = (S, f)$, f deterministic slices → states. View $V_L(t) = f(\mathcal{G}_t|_S)$, root $\mathsf{Com}(V_L(t))$; lens named $N(\texttt{lens}, L)$. A view is a state ⇒ sliceable, openable, composable, committable as a repo.

Composition. Selectors span contexts ⇒ a view may combine several apps and raw repos. Each box keeps its context's guarantees (what was accepted, under which rule, what was excluded) ⇒ a view is as trustworthy as its weakest source. **Exclusions only add:** a curation lens can hide more, never restore.

Prop 3.10 (Views are determined). All honest parties get the same root. *(Thm 4 + determinism + unique representation.)*

Def 3.11 (Pointwise). $\exists\,\tau, g$: $\tau(x, r)$ injective in x, $g(r) \in Rec \cup \{\bot\}$,

$$f(Z) = \{\tau(x, r) \mapsto g(r) : (x \mapsto r) \in Z,\ g(r) \neq \bot\}$$

with $x = (c, P, s, k)$.

Prop 3.12 (Pointwise update). If Z, Z' differ on keys X: delete $\tau(x, Z(x))$, then insert $\tau(x, Z'(x)) \mapsto g(Z'(x))$. Cost $2|X|$ ops × $2(h+1)$ nodes.

lens	$\tau(x, r)$	$g(r)$	notes
filter	x	r if type θ else $\bot$	pointwise
validation	x	r if $\varphi(r)$ else $\bot$	read-time check, anyone; single-record rules only
curation	x	r unless x in excluded repo	on top of the context's $\mathcal{X}$
index on a	$(r.a, x)$	r	value v = one opening, $O(h+j)$; top-j = last j keys
embedding	x	$E(r) \in \mathbb{R}^d$	E must be bit-deterministic (fixed-point, quantized)
sub-log (over $\mathcal{U}$)	$(\ell.P, j)$	ℓ	P's updates form an interval
aggregate	—	—	not pointwise, incremental $O(\lvert X\rvert)$
join / general	—	—	no succinct fraud proof

Def 3.14 (Key-revealing). Pointwise and x computable from $\tau(x, r)$. (Filter, validation, curation, index, sub-log, embedding.)

8. Indexers and fraud proofs

Def 3.13 (Claim). Signed $c = (\iota, N(\texttt{lens}, L), t, R)$ asserting $R = \mathsf{Com}(V_L(t))$ (log lenses: $t \to M$).

Point read at $x = (c, P, s, k)$ *as of* t: ReadAt in c + 3×ExclAt (if c is an app) + lines 6–8 on $[k, k]$ ⇒ $\mathcal{G}_t[c](P, s, k)$. (As of t, so later exclusions can't make honest claims "wrong".) Over a log: positional opening, or $\bot$ if $j > M$.

Fraud proof $fp = (y, V', \pi, x, p)$. $\mathsf{CheckFraud}(c, fp)$

```
1  require Check(R,[y,y],V',π)                    ▷ V' = ∅ or {y↦v}
2  if V' ≠ ∅: x ← key revealed by y
3  z ← value at x from point read p; require p checks as of t
4  U ← {y↦g(z)} if z≠⊥ ∧ g(z)≠⊥ ∧ τ(x,z)=y, else ∅
5  return U ≠ V'                                  ▷ true ⇒ claim wrong at y
```

Prop 3.15 (Succinct fraud proofs). L pointwise & key-revealing.

1. **Completeness:** $R \neq \mathsf{Com}(V_L(t))$ (well-formed inputs) ⇒ $\exists\, fp$ accepted; 1 opening + 1 point read + one eval of τ, g.
2. **Soundness:** $R = \mathsf{Com}(V_L(t))$ ⇒ no accepted fp, except opening binding / N binding / Prop 3.8. Holds for claims of **any age**.

Idea: $U = V|_{[y,y]}$ whenever z is true and ($y = \tau(x, z)$ or $y \notin V$). Completeness: pick y in the symmetric difference. Soundness: opening binding forces $V' = V|_{[y,y]} = U$.

Cost: log lens $O(h+1) + 2\lfloor \log_2 M \rfloor$; slice lens: map opening + 2 log openings (head) + ≤6 (exclusions) + 1 state opening.

Equivocation: two roots, same (L, t) ⇒ ≥1 wrong ⇒ fraud proof.

Discovery. $\mathcal{U}$ names every repo with a head; rule logs what passed each rule; app logs what each app accepted ⇒ "active publishers", "repos satisfying V", "apps carrying ρ" are pointwise lenses over logs ⇒ omissions are fraud-provable.

Trust ladder: trust → hold to account (one honest watcher sees root) → cross-check k indexers → verify proof of view → recompute. Watchers find y by walking two trees (L6.1), $O(h+1)$ per differing key. No challenge window.

9. Search

Score $\langle q, v\rangle$, $\|q\| \leq 1$. Three ways to serve, in order of preference:

Bundle (default for small views). The embedding view, sharded by key interval; each shard is an opening of the claimed root. Reader fetches all, ranks locally ⇒ **exact and complete** (opening binding: no forged *or omitted* entry); hosts learn which views, never the query. ~26 MB for 10^5 × 256-d int8; then follow by change.

Bounded tree (large views). Def 3.16. Leaves $(x, v) \in V$; internal nodes $(\mu, \varepsilon, \text{children})$. **Sound** iff every leaf v below has $\|v - \mu\| \leq \varepsilon$. Bound: $\beta(n) = \langle q, \mu\rangle + \varepsilon$ (internal), $\langle q, v\rangle$ (leaf).

$\mathsf{Search}(\alpha, q, j)$ — best-first: pop max-β node; stop if $|T| = j$ and $\beta(n) < \min_T \langle q, v\rangle$; leaves → keep top j; internal → push children.

Lemma 3.17 (Exact pruning).

$$\begin{aligned}\langle q, v\rangle &= \langle q, \mu\rangle + \langle q, v - \mu\rangle \\ &\leq \langle q, \mu\rangle + \|q\|\,\|v - \mu\| \leq \langle q, \mu\rangle + \varepsilon\end{aligned}$$

⇒ on a sound tree, Search returns the exact top-j. Quantization absorbed into ε.

Fraud proofs: **false bound** = path to leaf with $\|v - \mu\| > \varepsilon$; **forged leaf** = path + point read of embedding view. **Omission: open** (unlike the bundle). Insert $O(\mathit{depth} \cdot d)$; delete keeps soundness. Host sees which nodes are expanded (rough query location).

Answering queries — a downgrade: the indexer sees every query.

10. Instantiation

Ordering = blockchain. Blocks = sequence; timestamps = clock; consensus = Verify. Registry = contract(s) running Apply: stores $\mathcal{R}, \mathcal{A}, \mathcal{H}, \mathcal{V}, \mathcal{D}, \mathcal{X}, \mathcal{C}$; for $\mathcal{U}$ and each $\mathcal{L}$ only MMR peaks + length. May split: data registry ($\mathcal{H}, \mathcal{U}$) / rule registry ($\mathcal{R}, \mathcal{V}$, rule logs) / app registry (rest). ι = chain id + address (EIP-712 domain). Exclusions stored by hash: P, $(P, N(s))$, $(P, N(s), N(k))$; one counter slot per exclusion = flag (parity).

- **Validate / accept** check h against $\mathcal{H}(\rho)$ if latest, else a positional opening of $\mathcal{U}$ against its peaks ($\leq 2\lfloor \log_2 M \rfloor$ hashes).
- **Evidence costs** (estimates): on-chain check ∝ records read; pairing proof ~250k gas regardless of state size; attestation = signature (few k gas) from a key a program lists after checking hardware attestation once. Batch verifiers amortize; one validation serves all apps.
- **Gas** (estimates): update ~20k (first ~100k); acceptance ~20k (first ~100k) + witness; exclude/restore = counter + peak + length.
- Dir by **storage proofs** (lists: publisher → repos; app → accepted publishers → accepted repos; plus head/entry slots; no prover) or **checkpoints** (posted

$\mathsf{dir}_{A,t}$ + validity proof; $O((p+r)(h+1))$ nodes).

- Finality δ: ~1h full node on rollup; ~7 days for trustless light reader on optimistic rollup.

Com = **Merkle Search Tree.** $\mathrm{level}(k) = \lfloor \mathsf{lz}(N(k))/b \rfloor$, fanout 2^b. Level-ℓ node = keys of level exactly ℓ in a *gap* between consecutive keys of level $> \ell$ ⇒ tree is a function of the map (unique representation). Height h = max level; $h \approx \log_{2^b} n$; $h \le 2\lambda/b$. **Well-formed:** sorted, inside gap, level exactly ℓ, $\le B_{\max}$ keys.

Lemma 6.0 (Honest nodes small; ROM). $\Pr[\text{node} \ge m \text{ entries}] \le (1-2^{-b})^m$; $b=4$, $B_{\max}=1024$: $\approx 2^{-95}$. *(geometric run length)*

Shape attacks: height ℓ costs ~$2^{b\ell}$ tries; filling a node is free but $B_{\max}$ makes it *malformed* ⇒ hurts only the key-chooser's readers ⇒ one key-chooser per tree. App directories: keys chosen via α_A; if anyone with a proof may accept, filling a node costs ~$B_{\max}$ paid acceptances (unanalysed).

Lemma 6.1 (Locality). Maps differing on Δ keys share all but $\le 2\Delta(h+1)$ nodes per tree. *(a key lies in the closed gap of ≤ 2 nodes per level)* ⇒ update rewrites $\le 2(h+1)$; diff-walk $O(\Delta(h+1))$.

Lemma 6.2 (Openings). Opening on I (holding m keys) = nodes whose gaps meet I; count

$$\le (h+1) + \sum_{k \in D|_I} \mathrm{level}(k) \;\le\; (h+1)(m+1), \qquad \mathbb{E} = (h+1) + \tfrac{m}{2^b-1}.$$

Opening binding by induction over reached nodes (well-formedness confines keys to gaps).

LCom = **Merkle Mountain Range.** Leaf $N(\texttt{leaf}, l_j)$; node $N(\texttt{node}, L, R)$; $M = \sum 2^{e_i}$ ⇒ perfect trees, roots = peaks; root = $N(\texttt{mmr}, M, peaks)$; $\mathsf{st} = (M, peaks)$. Append = add 1-leaf tree, merge equal-size rightmost. Shape fixed by M ⇒ ungrindable. (On-chain: keccak over `abi.encode`, never `encodePacked`.)

Lemma 6.3 (MMR). (1) binding & position binding ⇐ N binding (tags separate leaf/node); (2) ≤ $\lfloor \log_2 M \rfloor + 1$ peaks (set bits of M); (3) opening ≤ $2\lfloor \log_2 M \rfloor$ names; (4) Δ appends ≤ $\Delta + \lfloor \log_2 M \rfloor$ merges ⇒ < 1 merge avg, ≤ $\lfloor \log_2 M \rfloor$ each.

11. Appendix A — agents and the semantic CDN (design)

- **One server for every app:** list apps + manifests (a record in the owner's repo: purpose, required rules, views with lens names and claims) → mount views → same tools on every view (describe, search, get, similar, count, browse). Works because every view is the same kind of state.
- **Semantic CDN** = route by meaning, deliver by name; routing is checkable, not trusted.

Def A.1 (Clustered view). Model E, fixed centres $C = (\mu_1, \dots, \mu_m)$ named in the lens:

$$\tau(x, r) = (\kappa(r), x), \quad g(r) = E(r), \quad \kappa(r) = \text{nearest centre (lowest index on ties)}$$

Pointwise + key-revealing ⇒ **a cluster = one interval = one opening, complete** (fixes the bounded tree's omission gap); misfiled / wrong / omitted entry ⇒ fraud proof (Prop 3.15); follows by change. New centres ⇒ new lens.

Def A.2 (Coverage). Claim + radii: $\mathsf{cov} = (\iota, N(\texttt{lens}, L), t, R, (\varepsilon_i))$. Sound iff every v in cluster i has $\|v - \mu_i\| \le \varepsilon_i$. Bound $\beta_i(q) = \langle q, \mu_i \rangle + \varepsilon_i$. A few KB per view.

Prop A.3 (Checkable). Unsound ⇒ one opening at one entry proves it; sound ⇒ $\langle q, v \rangle \le \beta_i(q)$ (L3.17). Loose-but-honest radius = cost, not fraud.

$\mathsf{Route}(q, j)$ — best-first over every cluster of every view by $\beta_i(q)$; stop when $|T| = j$ and next bound < j-th best; else open cluster (one checked opening from any host), keep top j.

Prop A.4 (Exact & complete). On sound coverages: exact top-j across all views; opens only clusters with bound ≥ the final j-th score. *(opening binding + A.3)*

Recursion. Directory = a view of clusters keyed $(N(\texttt{lens}, L), i)$ with value (μ_i, ε_i); cluster it again with upper radius

$$\delta = \max_i \left(\|\mu_i - \nu\| + \varepsilon_i \right)$$

(triangle inequality ⇒ sound if lower levels are) ⇒ one tree of clustered views: upper levels = views, lower = records. Many directories, each a curation; or start from a rule's log.

Guarantees: no authority · exactness · privacy (query local; hosts see clusters opened) · delivery by name from any host. **Cost:** nm inner products to route ($10^4 \times 8 \approx$ ms), then one opening per cluster opened.

- **Cartridge** = (lens name, t, root, fetched clusters/shards): hot-swappable, composable (same E ⇒ comparable scores), pinned (answers cite root + t), checked, followed by change. Agents publish their own memory as repos ⇒ cartridges owned by their key.
- **Open:** one bit-exact E per CDN (cross-model mapping not given); choosing centres / reclustering / radius tightness (cost only); cluster counts (aggregate, no succinct fraud proof); leakage from clusters opened; paid/sealed views.

12. Open / unproven

Non-pointwise fraud proofs · bit-deterministic embeddings (assumed) · on-chain cost of ν_V, α_A, χ_A (estimates only) · constructions of proofs/attestations for rules (named, not built) · app-directory shape under permissionless acceptance · epoch window delay · watcher sufficiency & collusion · bounded-tree omissions, search cost, query leakage · sub-log index / address grinding · nested-directory interval bound · joins · gas/proof sizes (estimates) · light-client protocol (assumed) · sealed data, licenses, accepting unreadable data (WP2).